

Trabalho: Gerenciador de Contatos com Árvore Binária em C

Objetivo:

Desenvolver um sistema em C para inserir, buscar, remover e imprimir contatos utilizando uma árvore binária de busca. Os contatos serão organizados pelo nome (a key será o nome).

Estrutura de Dados:

Crie uma estrutura chamada 'Contato' com os seguintes campos:

- nome (string de até 50 caracteres).
- celular (int).
- Ponteiros para os filhos esquerdo e direito.

Um nó deve ser implementado conforme apresentado abaixo:

```
typedef struct Contato {  
    char nome[50]; //key  
    int celular;  
    struct Contato* left;  
    struct Contato* right;  
} Contato;
```

Funcionalidades:

Implemente as seguintes operações na árvore binária:

1. *Inserir_Contato*
2. *Buscar_Contato (por nome).*
3. *Remover_Contato (por nome).*
4. *Imprimir_Contatos*

Menu Interativo:

Crie um menu no console que permita ao usuário escolher entre as opções das funcionalidades e também a função 'sair'.

Dicas:

- Divida o projeto em partes menores e teste cada funcionalidade antes de integrá-las.
- Adicione comentários no código para facilitar a compreensão.

Critério de Hierarquia:

Os nós da árvore serão organizados com base no 'nome':

- Se o nome do novo contato for lexicograficamente menor, ele será inserido à esquerda.
- Se o nome for maior, será inserido à direita.

Função para comparar dois nomes:

```
int compararNomes(char* nome1, char* nome2) {  
    return strcmp(nome1, nome2);  
}
```

Explicação da Função:

A função compararNomes utiliza strcmp da biblioteca <string.h>, que retorna:

- Um valor **negativo** se nome1 é **menor** que nome2.
- **Zero** se são **iguais**.
- Um valor **positivo** se nome1 é **maior** que nome2.

CONSIDERE QUE O PROGRAMA IRÁ RODAR COM NOMES SOMENTE EM MAIÚSCULO, OU SEJA, TESTEM DIGITANDO NOMES JÁ EM MAIÚSCULO (NÃO PRECISA FAZER TOUPPER PARA CADA CARACTER).

Código base:

```
//-----  
  
#include <stdio.h>  
  
#include <stdlib.h>  
  
#include <string.h>  
  
  
// Estrutura para um contato  
typedef struct Contato {  
    char nome[50];  
    char celular[15];  
    struct Contato* esquerda;  
    struct Contato* direita;  
} Contato;  
  
  
// Funções para implementar no final do código  
Contato* inserir(Contato* raiz, Contato* novo);  
Contato* buscar(Contato* raiz, const char* nome);  
Contato* remover(Contato* raiz, const char* nome);  
void listar(Contato* raiz);  
void menu();  
  
int main() {  
    Contato* raiz = NULL;  
    int opcao;  
    do {  
        menu();  
        scanf("%d", &opcao);  
  
        switch (opcao) {  
            case 1: {  
                // Adicionar contato  
                Contato* novo = (Contato*)malloc(sizeof(struct Contato));  
                printf("Nome: ");
```

```

scanf("%s", novo->nome);
printf("Celular: ");
scanf("%s", novo->celular);
novo->esquerda = NULL;
novo->direita = NULL;
raiz = inserir(raiz, novo);
break;
}
case 2: {
    // Buscar contato
    char nome[50];
    printf("Nome para buscar: ");
    scanf("%s", nome);
    Contato* encontrado = buscar(raiz, nome);
    if (encontrado) {
        printf("Contato encontrado: %s, Celular: %s\n", encontrado->nome, encontrado->celular);
    } else {
        printf("Contato não encontrado.\n");
    }
    break;
}
case 3: {
    // Remover contato
    char nome[50];
    printf("Nome para remover: ");
    scanf("%s", nome);
    raiz = remover(raiz, nome);
    break;
}
case 4: {
    // Imprimir contatos
    printf("Contatos:\n");
    ImprimirArvoreDeContatos(raiz);

```

```
        break;
    }
    case 5:
        printf("Saindo...\n");
        break;
    default:
        printf("Opção inválida! Tente novamente.\n");
    }
} while (opcao != 5);
return 0;
}
```

// Funções para implementar

```
Contato* inserir(Contato* raiz, Contato* novo) {
    // Implementar lógica de inserção
    return raiz;
}
```

```
Contato* buscar(Contato* raiz, const char* nome) {
    // Implementar lógica de busca
    return NULL;
}
```

```
Contato* remover(Contato* raiz, const char* nome) {
    // Implementar lógica de remoção
    return raiz;
}
```

```
void ImprimirArvoreDeContatos(Contato* raiz) {
    // Implementar lógica de listagem
}
```

```
void menu() {  
    printf("\nMenu:\n");  
    printf("1. Adicionar contato\n");  
    printf("2. Buscar contato\n");  
    printf("3. Remover contato\n");  
    printf("4. Listar contatos\n");  
    printf("5. Sair\n");  
    printf("Escolha uma opção: ");  
}
```